

Fall 2024

Ali Sharifi-Zarchi

CE Department
Sharif University of Technology

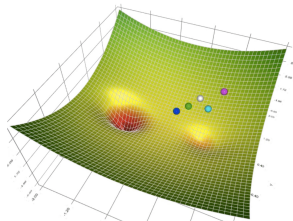
October 22, 2024



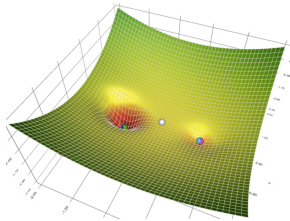
4 References

How to Update Weights?

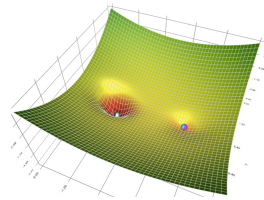
- Imagine training a large model like ChatGPT. It has billions of parameters that need to be adjusted.
- If we used **random search** to update these weights, it would take an astronomical number of trials to find good parameters.
- How to make training feasible at this massive scale?
- Image adapted from Medium: AdaGrad Optimization Algorithm



Visualizing parameter space with
random initial points.



Random search approach in
high-dimensional space.



Challenges in finding optimal weights using random search.

How to Update Weights?

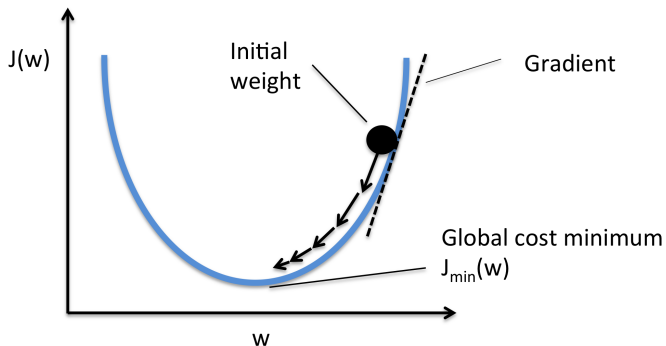
Options for Updating Weights:

- **Random Search:** Tries values randomly—inefficient and impractical.
- **Gradient Descent:** Follows the slope of the loss function—efficient and guided.

Why Gradient Descent?

- It updates weights by following the slope, reducing error with each step.
- Controlled, stepwise updates ensure we move closer to minimizing the loss effectively.

How to Update Weights?



Gradient descent: adjusting weights to minimize cost by following the gradient.

Gradient optimization image

Gradient Descent: Concept and Weight Updates

Gradient Descent: Minimizes the loss function by updating weights based on the gradient.

Weight Update Rule:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial L}{\partial w}$$

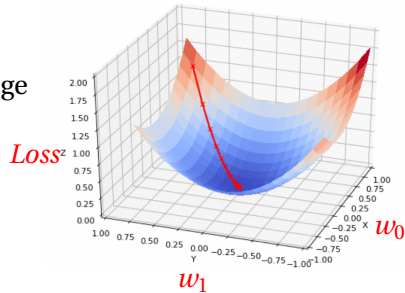
Where:

- η is the learning rate (step size).
- $\frac{\partial L}{\partial w}$ is the gradient of the loss function with respect to w .

Weight Update Formula:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial L}{\partial w}$$

- Compute the gradient of the loss function.
- Update the weights using the update rule.
- Repeat until convergence.
- Image adapted from Data Science Stack Exchange



2 Backpropagation

Vectorized Backpropagation

4 References

① Gradient Descent

② Backpropagation

Forward and Backward Passes

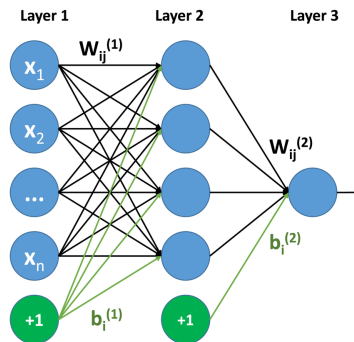
Vectorized Backpropagation

③ Foundations in Detail: Initialization, Loss, and Activation

④ References

Multi-Layer Neural Network

The diagram below illustrates a three-layer neural network

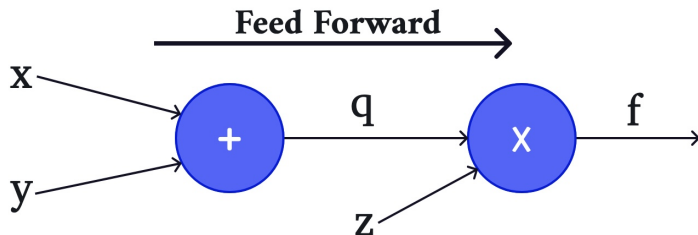


Neural Network

How should $\frac{\partial L}{\partial w}$ be calculated for each weight w in this neural network?

$$f(x, y, z) = (x + y)z$$

$$f(x, y, z) = (x + y)z$$



A Simple Example: Forward Pass

Function:

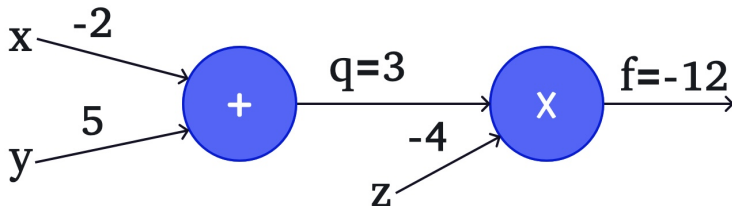
$$f(x, y, z) = (x + y)z$$

Example:

$$x = -2, \quad y = 5, \quad z = -4$$

Steps:

- $q = x + y = 3$
- $f = q \times z = -12$



Backpropagation

A more efficient method:

- Use the chain rule to compute all gradients in one backward pass.
- This method avoids changing each weight separately.

Advantages:

- Complexity is reduced to $\Theta(n)$.
- Much faster, especially for large neural networks.

Chain Rule for Gradients

The **chain rule** helps us find the gradient of a function that is composed of other functions.

Example:

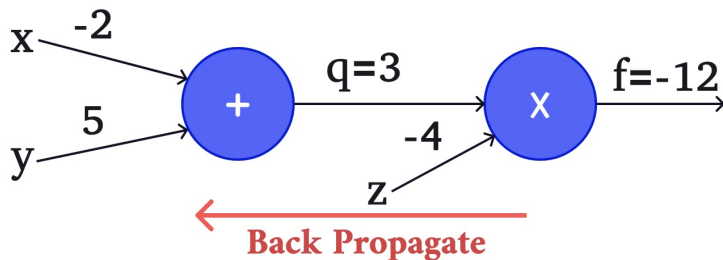
$$z = f(g(x))$$

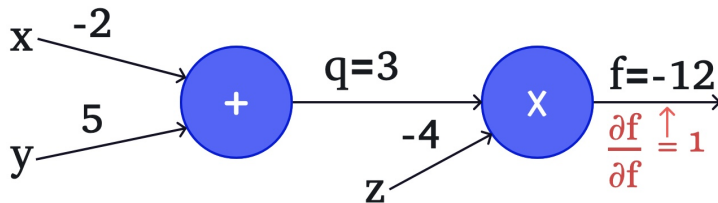
- f is a function of $g(x)$
- $g(x)$ is a function of x

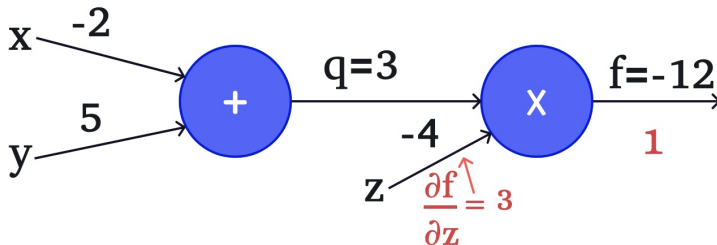
To find $\frac{\partial z}{\partial x}$, we use the chain rule:

$$\frac{\partial z}{\partial x} = \frac{\partial f}{\partial g} \times \frac{\partial g}{\partial x}$$

This means we multiply the gradient of the outer function by the gradient of the inner function.

$$f(x, y, z) = (x + y)z$$
$$x = -2, \quad y = 5, \quad z = -4$$


$$f(x,y,z) = (x+y)z$$
$$x = -2, \quad y = 5, \quad z = -4$$
$$\frac{\partial f}{\partial f} = 1$$


$$f(x, y, z) = (x + y)z$$
$$x = -2, \quad y = 5, \quad z = -4$$
$$f = qz, \quad \frac{\partial f}{\partial z} = q = 3$$


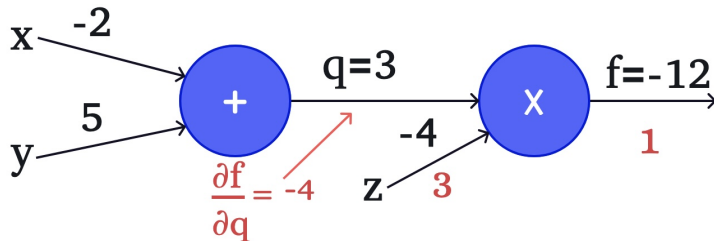
$$f(x, y, z) = (x + y)z$$

$$f(x, y, z) = (x + y)z$$

$$x = -2, \quad y = 5, \quad z = -4$$

$$f = qz, \quad \frac{\partial f}{\partial z} = q = 3$$

$$\frac{\partial f}{\partial q} = z = -4$$



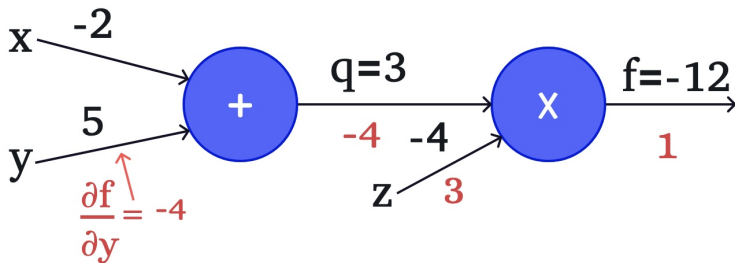
$$f(x, y, z) = (x + y)z$$

$$f(x, y, z) = (x + y)z$$

$$x = -2, \quad y = 5, \quad z = -4$$

$$q = x + y, \quad \frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y} = -4 \cdot 1 = -4$$



$$f(x, y, z) = (x + y)z$$

$$f(x, y, z) = (x + y)z$$

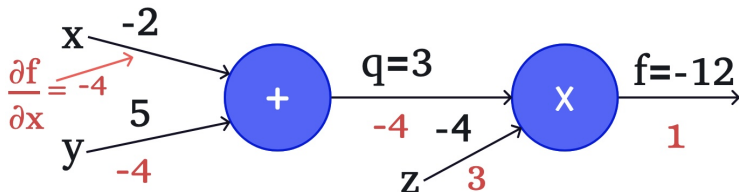
$$x = -2, \quad y = 5, \quad z = -4$$

$$x = -2, \quad y = 5, \quad z = -4$$

$$q = x + y, \quad \frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1$$

$$q = x + y, \quad \frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x} = -4 \cdot 1 = -4$$



$$f(x, y, z) = (x + y)z$$

$$f(x, y, z) = (x + y)z$$

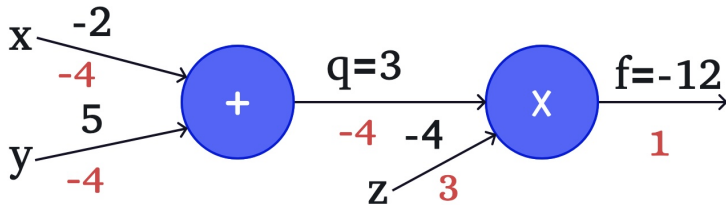
$$x = -2, \quad y = 5, \quad z = -4$$

$$x = -2, \quad y = 5, \quad z = -4$$

$$q = x + y, \quad \frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1$$

$$q = x + y, \quad \frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x} = -4 \cdot 1 = -4$$



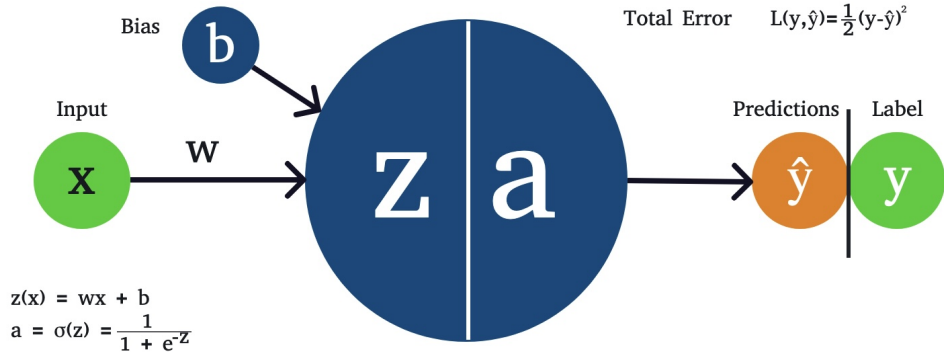
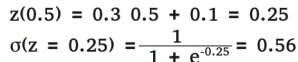
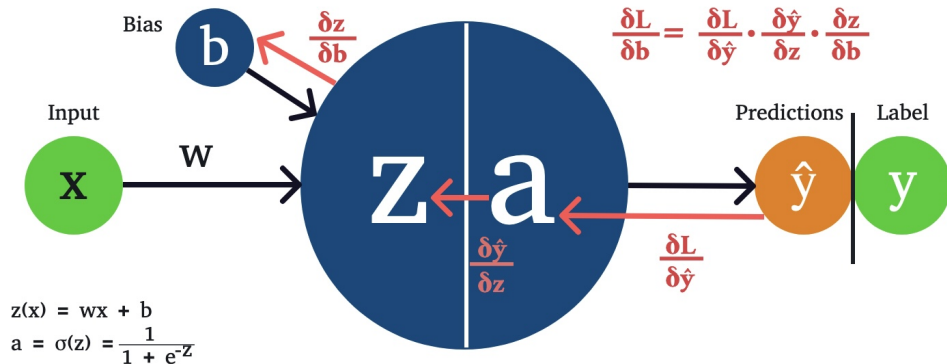


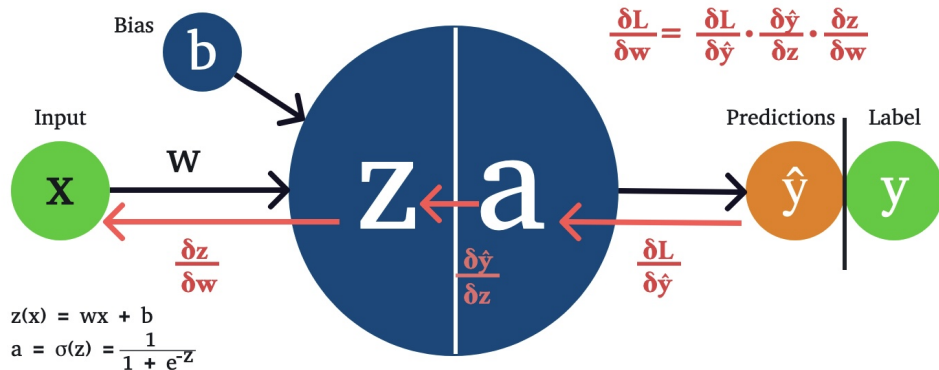
Image adapted from “Deep learning backpropagation” Web article



Backward Pass



Backward Pass



Backward Pass



① Gradient Descent

② Backpropagation

Forward and Backward Passes

Vectorized Backpropagation

③ Foundations in Detail: Initialization, Loss, and Activation

④ References

Derivative of a Vector by a Vector: leveraging matrix operations for faster computation.

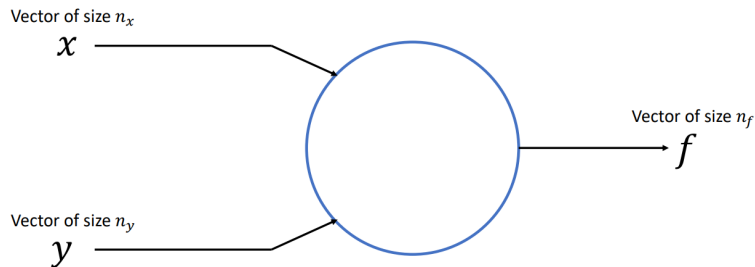
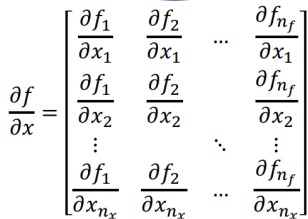
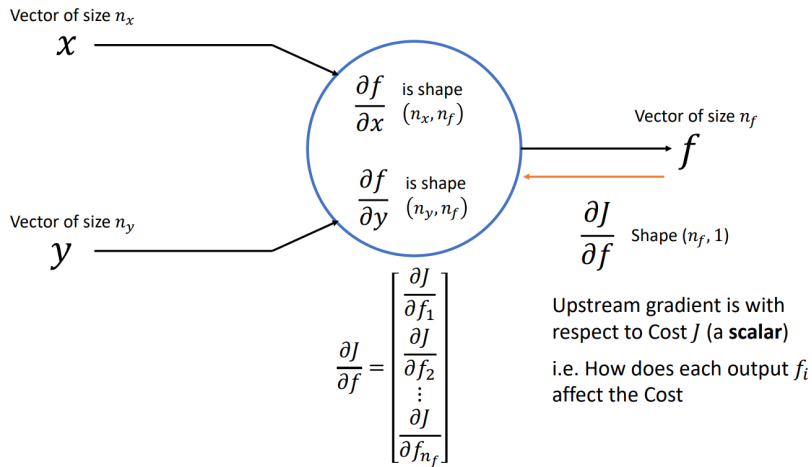


Image adapted from Elec502 vectorized backpropagation slides

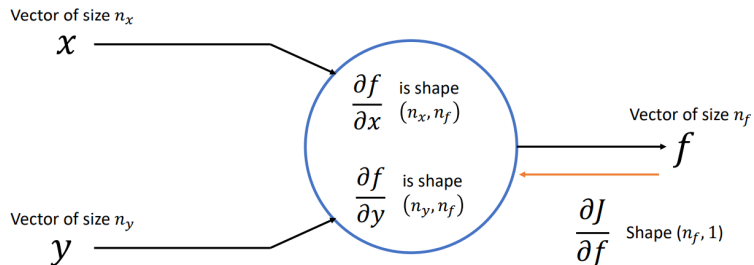
Local Derivatives are Jacobian Matrices



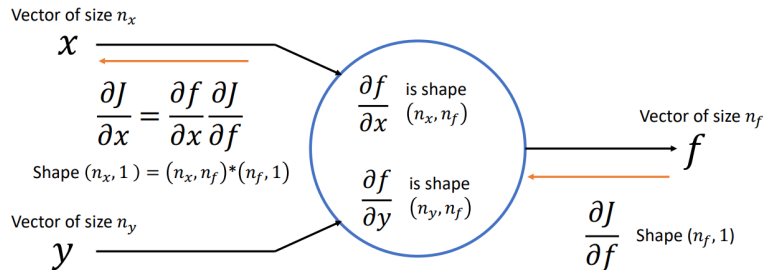
Vectorized Backpropagation



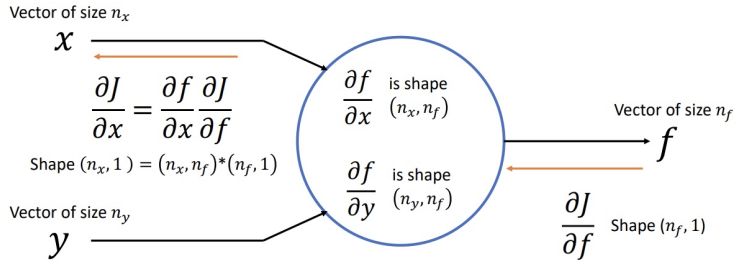
Vectorized Backpropagation



Apply chain rule like before!



Applying the chain rule involves matrix-vector multiplication



$$\frac{\partial J}{\partial x} = \frac{\partial f}{\partial x} \frac{\partial J}{\partial f} \rightarrow \begin{bmatrix} \frac{\partial J}{\partial x_1} \\ \frac{\partial J}{\partial x_2} \\ \vdots \\ \frac{\partial J}{\partial x_{n_x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}, \frac{\partial f_2}{\partial x_1}, & \dots & \frac{\partial f_{n_f}}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial x_{n_x}}, \frac{\partial f_2}{\partial x_{n_x}} & \dots & \frac{\partial f_{n_f}}{\partial x_{n_x}} \end{bmatrix} \begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$$

Chain Rule – Matrix-Vector Multiply

$$\frac{\partial J}{\partial x} = \frac{\partial f}{\partial x} \frac{\partial J}{\partial f} \rightarrow \begin{bmatrix} \frac{\partial J}{\partial x_1} \\ \frac{\partial J}{\partial x_2} \\ \vdots \\ \frac{\partial J}{\partial x_{n_x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}, \frac{\partial f_2}{\partial x_1}, & \dots & \frac{\partial f_{n_f}}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial x_{n_x}}, \frac{\partial f_2}{\partial x_{n_x}} & \dots & \frac{\partial f_{n_f}}{\partial x_{n_x}} \end{bmatrix} \begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$$

$$\text{Shape } (n_x, 1) = (n_x, n_f) * (n_f, 1)$$

Chain Rule – Matrix-Vector Multiply

$$\frac{\partial J}{\partial x} = \frac{\partial f}{\partial x} \frac{\partial J}{\partial f} \rightarrow \begin{bmatrix} \frac{\partial J}{\partial x_1} \\ \frac{\partial J}{\partial x_2} \\ \vdots \\ \frac{\partial J}{\partial x_{n_x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}, \frac{\partial f_2}{\partial x_1}, & \dots & \frac{\partial f_{n_f}}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial x_{n_x}}, \frac{\partial f_2}{\partial x_{n_x}} & \dots & \frac{\partial f_{n_f}}{\partial x_{n_x}} \end{bmatrix} \begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$$

Jacobian

Shape $(n_x, 1) = (n_x, n_f) * (n_f, 1)$

Chain Rule – Matrix-Vector Multiply

The diagram illustrates the chain rule for backpropagation through a Jacobian matrix. It shows the relationship between the total derivative of the loss J with respect to an input x and the product of the Jacobian matrix and the upstream gradient.

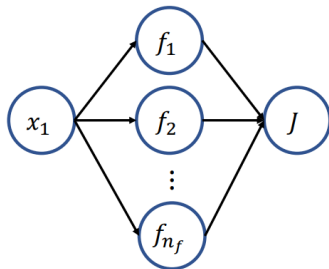
The equation shown is:

$$\frac{\partial J}{\partial x} = \frac{\partial f}{\partial x} \frac{\partial J}{\partial f} \rightarrow \begin{bmatrix} \frac{\partial J}{\partial x_1} \\ \frac{\partial J}{\partial x_2} \\ \vdots \\ \frac{\partial J}{\partial x_{n_x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}, \frac{\partial f_2}{\partial x_1}, & \dots & \frac{\partial f_{n_f}}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial x_{n_x}}, \frac{\partial f_2}{\partial x_{n_x}} & \dots & \frac{\partial f_{n_f}}{\partial x_{n_x}} \end{bmatrix} \begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$$

Annotations in the diagram include:

- Jacobian**: A blue arrow points from this label to the Jacobian matrix.
- Upstream Gradient**: An orange arrow points from this label to the vector $\begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$.
- Shape**: The text $\text{Shape } (n_x, 1) = (n_x, n_f) * (n_f, 1)$ is shown below the matrix, indicating the dimensions of the vectors and the matrix multiplication.

Chain Rule – Matrix-Vector Multiply

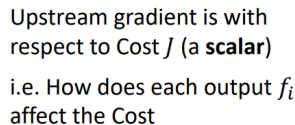


$$\frac{\partial J}{\partial x} = \frac{\partial f}{\partial x} \frac{\partial J}{\partial f} \rightarrow$$

$$\begin{bmatrix} \frac{\partial J}{\partial x_1} \\ \frac{\partial J}{\partial x_2} \\ \vdots \\ \frac{\partial J}{\partial x_{n_x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}, \frac{\partial f_2}{\partial x_1}, & \dots & \frac{\partial f_{n_f}}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial x_{n_x}}, \frac{\partial f_2}{\partial x_{n_x}} & \dots & \frac{\partial f_{n_f}}{\partial x_{n_x}} \end{bmatrix} \begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$$

$$\text{Shape } (n_x, 1) = (n_x, n_f) * (n_f, 1)$$

$$\frac{\partial J}{\partial x_1} = \frac{\partial f_1}{\partial x_1} \frac{\partial J}{\partial f_1} + \frac{\partial f_2}{\partial x_1} \frac{\partial J}{\partial f_2} + \dots + \frac{\partial f_{n_f}}{\partial x_1} \frac{\partial J}{\partial f_{n_f}}$$

[illegible]

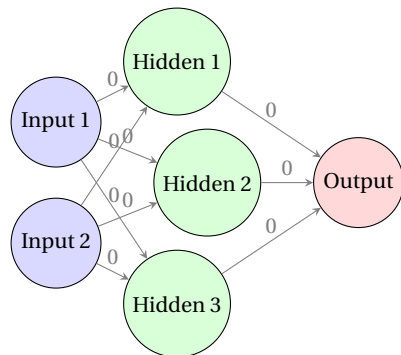
- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

Weight Initialization

Example: Imagine a network where all weights are initialized to zero.

Issue: If all weights are zero, each neuron in a layer will produce **identical** outputs. This symmetry prevents the network from learning **distinct features**, as every neuron updates identically.

Solution: To break this symmetry, weights need to be initialized with small random values, allowing neurons to learn unique features and avoid identical updates.



All weights initialized to zero

Why Weight Initialization Matters

Importance:

- Proper **initialization** ensures **faster convergence** and improves **training stability**.
- Prevents issues like **vanishing** or **exploding gradients**, which can make training slow or unstable.

Question: How can we initialize weights to maximize **learning efficiency** and prevent gradient problems?

Zero Initialization and Random Initialization

Zero Initialization

- **Description:** Set all weights to zero.
- **Key Point:** Rarely used, as it leads to identical updates for all neurons, preventing the network from learning distinct features.

Random Initialization

- **Description:** Assign small random values to weights.
- **Distribution:** Typically, weights are initialized using a uniform or normal distribution.

$$w \sim \mathcal{U}(-\epsilon, \epsilon) \quad \text{or} \quad w \sim \mathcal{N}(0, \sigma^2)$$

- **Key Point:** Helps break symmetry but can still cause issues with gradient magnitudes.

Xavier Initialization

Description: Xavier Initialization is designed to keep the variance of activations consistent across layers, ideal for **sigmoid** and **tanh** activations.

Objective: Prevents the shrinking or exploding of signal magnitudes during forward and backward propagation.

Condition:

$$\frac{1}{n_l} \text{Var}[w] = 1$$

where n_l is the number of neurons in layer l .

Initialization Scheme:

$$w \sim \mathcal{U}\left(-\sqrt{\frac{1}{n_l}}, \sqrt{\frac{1}{n_l}}\right)$$

This results in a uniform distribution within the range $-\sqrt{\frac{1}{n_l}}$ to $\sqrt{\frac{1}{n_l}}$, ensuring stable signal variance across layers.

Condition:

$$\frac{1}{2}n_l \text{Var}[w] = 1$$

Initialization Scheme:

$$w_l \sim \mathcal{N}\left(0, \frac{2}{n_l}\right)$$

This implies a zero-centered Gaussian distribution with a standard deviation of $\sqrt{\frac{2}{n_l}}$, where biases are initialized to 0.

Choosing the Right Initialization – Examples

- **Scenario 1:** Using ReLU activation functions in a deep network.
 - **Best Choice:** He Initialization.
 - **Reason:** Helps maintain gradient flow through the layers.
- **Scenario 2:** Using Sigmoid activation functions in a shallow network.
 - **Best Choice:** Xavier Initialization.
 - **Reason:** Keeps variance balanced, which is crucial for non-ReLU activations.

Mean Squared Error (MSE)

Definition:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N \left(y^{(i)} - \hat{y}^{(i)} \right)^2$$

Characteristics:

- Amplifies larger errors due to squaring, making it sensitive to outliers.

Example of MSE Calculation

Example:

- Predicted values: $\hat{y} = [4.2, 3.8, 5.1]$
- True values: $y = [5.0, 4.0, 4.9]$
- Calculation:

$$\begin{aligned}\text{MSE} &= \frac{1}{3} [(5.0 - 4.2)^2 + (4.0 - 3.8)^2 + (4.9 - 5.1)^2] \\ &= \frac{1}{3} [0.64 + 0.04 + 0.04] = \frac{0.72}{3} = 0.24\end{aligned}$$

Binary Classification Loss – Binary Cross-Entropy

Binary Cross-Entropy:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{n} \sum_{i=1}^n \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

Example:

- Predicted probabilities: $\hat{y} = [0.7, 0.3, 0.9]$
- True labels: $y = [1, 0, 1]$

$$\begin{aligned}\mathcal{L}_{\text{BCE}} &= -\frac{1}{3} [1 \cdot \log(0.7) + (1-1) \cdot \log(1-0.7) \\ &\quad + 0 \cdot \log(0.3) + (1-0) \cdot \log(1-0.3) + 1 \cdot \log(0.9) + (1-1) \cdot \log(1-0.9)] \\ &= -\frac{1}{3} (\log(0.7) + \log(0.7) + \log(0.9)) \approx -\frac{1}{3} (-0.357 + -0.357 + -0.105) \approx 0.273\end{aligned}$$

Used to minimize the error between predicted probabilities and binary labels.

Categorical Cross-Entropy

Categorical Cross-Entropy Formula:

$$\mathcal{L}_{CCE} = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_c^{(i)} \log(\hat{y}_c^{(i)})$$

One-Hot Encoding

One-hot encoding represents categorical variables as binary vectors, with a 1 indicating the actual class and 0s elsewhere.

Example:

- Class 1: $[1, 0, 0]$
- Class 2: $[0, 1, 0]$
- Class 3: $[0, 0, 1]$

$$\hat{y} = \underbrace{\begin{bmatrix} 0.1 & 0.7 & 0.2 \\ 0.6 & 0.3 & 0.1 \\ 0.1 & 0.6 & 0.3 \end{bmatrix}}_{\text{Classes}} \quad \text{Samples}$$

Solution

- ① True labels:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- ② Calculation:

$$\mathcal{L}_{CCE} = -\frac{1}{3} (\log(0.7) + \log(0.6) + \log(0.3))$$

- ③ Compute log terms:

$$\log(0.7) \approx -0.357, \quad \log(0.6) \approx -0.511, \quad \log(0.3) \approx -1.204$$

$$\mathcal{L}_{CCE} = \frac{1}{3} \times 2.072 \approx 0.691$$

Limitation of Linear Activation

Why Transform Outputs? Raw outputs need to be transformed into meaningful values, such as probabilities.

The Problem: Linear activation lacks non-linearity, restricting the model to simple linear relationships.



Neural Networks: Why is the Max Operator Important?

- **Before:** Linear score function:

$$f = Wx$$

- **Now:** 2-layer Neural Network:

$$f = W_2 \max(0, W_1 x)$$

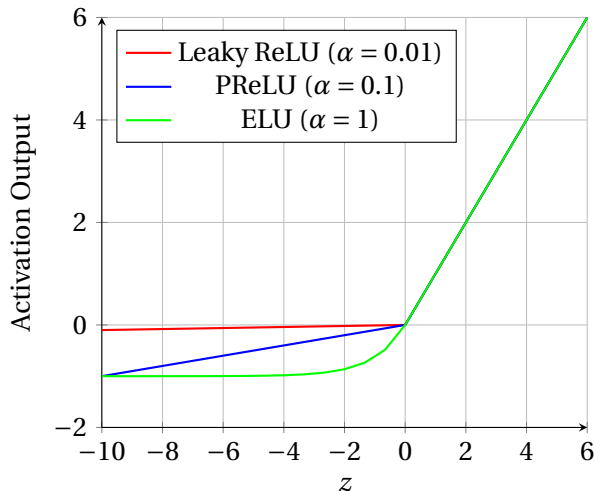
- The function $\max(0, z)$ is called an activation function (in this case, ReLU).
- Q: What if we try to build a neural network without an activation function?

$$f = W_2 W_1 x$$

$$W_3 = W_2 W_1 \in \mathbb{R}^{C \times H}, \quad f = W_3 x$$

- **A:** We end up with a linear classifier again!

Leaky ReLU, PReLU, and ELU Activation Functions

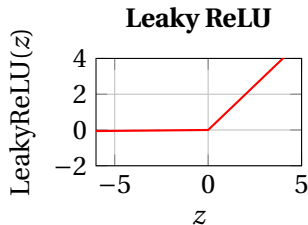


Variants of ReLU: Leaky ReLU

- Allows a small, non-zero gradient for negative inputs.

$$\text{LeakyReLU}(z) = \max(\alpha z, z), \quad \alpha = 0.01$$

- Helps prevent the "dead ReLU" problem, where neurons stop updating.



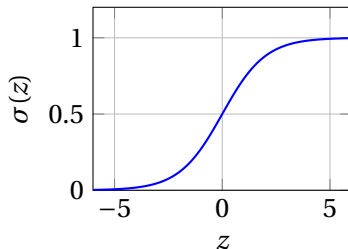
Leaky ReLU: Allows a small, non-zero gradient for negative inputs.

Sigmoid

Characteristics of Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

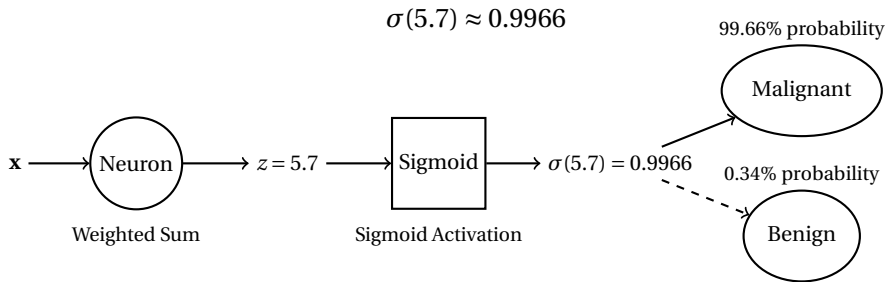
- $\sigma'(z) = \sigma(z)(1 - \sigma(z))$
- Squashes the input between 0 and 1, which makes it useful in probabilistic interpretations (e.g., logistic regression).
- Often used in output layers for binary classification problems.



Classification: Tumor Detection (Malignant vs. Benign)

Sigmoid Activation: Useful for binary classification!

- Example: For $z = 5.7$,



Tanh (Hyperbolic Tangent)

Characteristics of Tanh:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- Squashes input between -1 and 1, making it zero-centered (Balanced Updates → Reduced Bias in Gradient Descent → Faster Convergence)

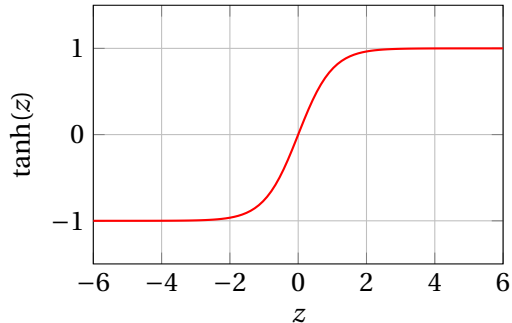
Advantages of Tanh:

- **Zero-Centered:** Output ranges from -1 to 1, making optimization easier.
- Better for **hidden layers** than Sigmoid due to zero-centered output.

Limitations:

- Similar saturation issues as Sigmoid: large input values push gradients towards zero (vanishing gradient problem).

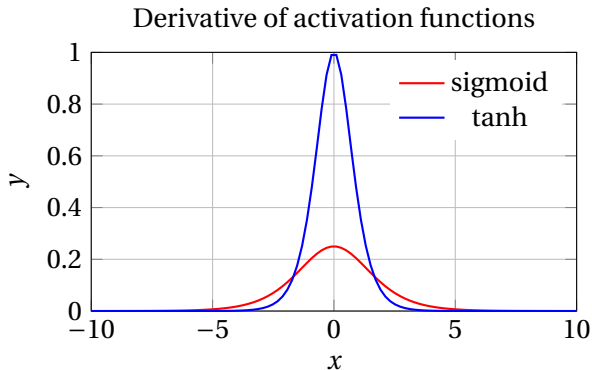
Tanh (Hyperbolic Tangent)



Question:

- How does Tanh help with faster convergence compared to Sigmoid?

Comparison: Sigmoid vs Tanh



- The derivative of the Tanh function has a much steeper slope at $x = 0$, meaning it provides a larger gradient for backpropagation compared to the Sigmoid function.

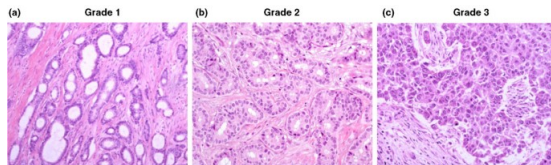
Problem: Multi-Class Tumor Classification

Scenario: We want to classify a tumor into one of three categories:

- **Class 0:** Benign
- **Class 1:** Malignant
- **Class 2:** Pre-cancerous

Goal: Given a set of tumor features, predict which class the tumor belongs to.

This is a **multi-class classification problem**, and we will use the **Softmax activation function** to assign probabilities to each class.



Microscopic images of tumor tissue, classified into grades representing different severity levels. Image adapted from Visual Analytics in Digital Computational Pathology

Example: Softmax Probabilities

Step 3: Calculate Softmax probabilities for each class

$$P(\text{Benign}) = \frac{4.48}{7.32} \approx 0.612, \quad P(\text{Malignant}) = \frac{2.23}{7.32} \approx 0.305, \quad P(\text{Pre-Cancerous}) = \frac{0.61}{7.32} \approx 0.083$$

Step 4: Make a classification decision

- The highest probability is 0.612 for the **Benign** class.
- Therefore, the model predicts that the tumor is **Benign** (Class 0).

- ## 4 References

Contributions

- **This slide has been prepared thanks to:**
 - Donya Navabi
 - Mohammad Aghaei
 - Sogand Salehi

- [1] T. Networks, “Benign vs malignant tumors.” *Web article*, 2023.
<https://www.technologynetworks.com/cancer-research/articles/benign-vs-malignant-tumors-364765>.
- [2] B. Central, “Breast cancer research article.” *Journal article*, 2010.
<https://breast-cancer-research.biomedcentral.com/articles/10.1186/bcr2607>.
- [3] S. Raschka, “Gradient optimization image.” *Image*, 2020.
<https://sebastianraschka.com/images/faq/gradient-optimization/ball.png>.
- [4] Xnought, “Backpropagation explainer.” *Website*, 2022.
<https://xnought.github.io/backprop-explainer/>.
- [5] Datamapu, “Deep learning backpropagation.” *Web article*, 2021.
https://datamapu.com/posts/deep_learning/backpropagation/.

- https://www.researchgate.net/figure/Our-neural-network-settings-weight-matrix-W-and-bias-b-are-recalibrated-fig1_335468765.

Any Questions?